Local NLP Toolkit

Ground rules

- Run **locally on CPU** using Hugging Face **pipelines** (same packages as in the lesson).
- You may consult model cards on the Hub, but **write your own code**.
- Keep prompts and decoding choices **deterministic** unless the task asks you to compare.

✅ **Task 1**

Goal: prove you can load pipelines, design a prompt, and interpret outputs.

Build a script mini_nlp.py that does **both**:

1. **Instruction rewrite (FLAN-T5-base)**
    - Input: one English sentence from the user (read from input() or a CLI flag).
    - Output: a **simpler English** rewrite that is **exactly one sentence**, **8–24 words**, and **ends with a period**.
    - Use **deterministic decoding** (no sampling).
    - If constraints aren't met, print a clear note (e.g., "constraint not satisfied").
2. **Sentiment (DistilBERT SST-2)**
    - Input: **three** sentences (ask the user for them or accept a comma-separated string).
    - Output: label + score for each.
    - After printing, add a one-paragraph note explaining **why there is no Neutral** in this model and what that implies.

⚡ Task 2 (Challenge)

Goal: batch processing + decoding comparison + basic measurement.

Extend your tool (new script or upgrade the first) to support **batch input from a .txt file** and **export a .csv**.

Pick **one** generation task:

- **A) Rewrite** (simplify) with FLAN-T5-base, **or**
- **B) One-sentence summarization** (DistilBART CNN) from 2–3 sentence inputs.

Requirements:

1. Accept --infile lines.txt (each line = one input).
2. Run **two decoding modes** on the same inputs:
    - **Beam search** (deterministic).
    - **Sampling** (choose temperature/top-p you think are reasonable).
3. Export results.csv with columns:
   input, output, decoding, tokens_out, constraint_passed, notes
    - tokens_out: count by your own function (approximate is fine).
    - constraint_passed: boolean (your validator).
    - notes: brief reason if failed (e.g., “>1 sentence”, “no period”, “too short”).
4. At the end, print a **mini-report**: for each decoding mode,
    - % of rows that passed constraints,
    - average tokens_out,
    - one sentence reflecting on **quality vs control**.

🚀 **Task 3 (Extra challenge — optional stretch)**

Goal: simple **guardrails + evaluation** across **three** tasks.

Add a small **"guardrail layer"** and a **metrics summary**:

1. Support **three tasks**:
    - Rewrite (FLAN-T5-base),
    - One-sentence summarization (DistilBART CNN),
    - Sentiment (DistilBERT).
2. Implement a **retry policy** for the two generation tasks:
    - If constraints fail on beam, **retry once** with sampling (different temperature/top-p).
    - Record whether retry fixed it.
3. Add a simple **neutral band** for sentiment: map scores near 0.5 (±0.1 by default) to **NEUTRAL** and report both the raw model label/score and your neutralized label.
4. Print a final **metrics block**:
    - Constraint pass-rate (beam vs sampling vs retry).
    - Avg output length (tokens or words).
    - Sentiment label distribution before/after neutral band.